

Week 2 — Day 1 (Sunday): Classes, Constructors, and Encapsulation

Object-Oriented Design | Theme: Track & Movement Control

Last week your track lived across parallel lists — the id at index 3 had to match the speed at index 3. The day someone removes a track from one list and forgets the other, everything falls apart. Today we make that bug impossible: the id, speed, heading, and status become **one object** that cannot fall out of sync. You will write your first class, give it a constructor that builds it correctly, and guard its data with properties.

Morning Block — SQL (09:00 – 09:40)

Topic: aggregates — **COUNT**, **SUM**, **AVG**, **GROUP BY**, and **HAVING** on the **classicmodels** database, inside the MySQL container.

Last week you did **SELECT** / **WHERE** / **ORDER BY** / **NULL** on single rows. Today the new idea is the opposite direction: collapsing **many rows into one** with grouping and aggregation. Your instructor will start (or re-run) the **classicmodels** container.

Goal first, syntax second. Try each goal yourself before reaching for a reference — this block is learning-by-practice.

Step 1 — Connect (5 min)

```
docker exec -it mysql-classic mysql -u root -psecret
```

```
USE classicmodels;  
SHOW TABLES;
```

Step 2 — COUNT + GROUP BY (10 min)

Goal: how many customers are in each country? One row per country, with a count, the largest count first.

(You will need a function that counts rows, a clause that groups rows by a column, and ordering. Find them.)

Step 3 — SUM and AVG per group (10 min)

Goal: for each customer (**customerNumber**) in the **payments** table, the total amount paid and the average amount paid. One row per customer.

Step 4 — GROUP BY + HAVING (10 min)

Goal: which product lines have more than 30 products? Show the product line and its product count, only for lines above that threshold.

Then push further — average `buyPrice` per product line, but **only** for lines whose average `buyPrice` is above 50.

Question to answer in your own words: you can filter rows *before* grouping and you can filter groups *after* aggregating. One of these uses **WHERE**, the other needs a different keyword. Which is which, and why can't you put an aggregate like `AVG(...)` in a **WHERE** clause? (Try it and read the error — the error is the lesson.)

Note: grouping is the same instinct you will use this afternoon in code — taking many separate things and treating them as belonging together.

Daily Goals

- Define a **class** and create objects from it with **new**
 - Explain the difference between a class (the blueprint) and an object (the instance)
 - Write a constructor and prove to yourself when it runs
 - Use constructor overloading and constructor chaining with **this(...)**
 - Make fields private and expose them through properties with validation
 - Override **ToString()** so an object prints itself
 - Explain encapsulation in your own words and point to where you built it
-

Schedule

Time	Block	Activity
09:00 – 09:40	Morning	SQL block (above)
09:40 – 10:15	Opening	Instructor — from parallel lists to one object; the constructor
10:30 – 11:30	Keywords, self-investigation, examples, comprehension	Read links, fill keyword table, answer the "why" questions, read and run the code examples, answer the comprehension questions
11:30 – 12:45	Long exercise — planning	Start the daily exercise; this block is mostly planning your class
14:45 – 17:45	Long exercise — build	Continue building the daily coding exercise

Keywords (10:30 – 11:30)

Term	Definition	Example
class	A blueprint for a type that bundles data and behavior together	<code>class Track { ... }</code>
object / instance	A concrete thing built from a class	<code>new Track(17)</code> is one object

Term	Definition	Example
<code>new</code>	Creates an object and runs its constructor	<code>Track t = new Track(17);</code>
field	A variable that belongs to an object	<code>private double _heading;</code>
constructor	A special method that runs when an object is created	<code>public Track(int id) { ... }</code>
default constructor	The free parameterless constructor — gone once you write your own	<code>new Track()</code>
constructor overloading	Several constructors with different parameters	<code>Track(int id)</code> vs <code>Track(int id, double speed)</code>
constructor chaining	One constructor calling another with <code>this(...)</code>	<code>Track(int id) : this(id, 0.0) { }</code>
property	A guarded way to read/write a field, with <code>get/set</code>	<code>public double Speed { get; set; }</code>
auto-property	A property that creates its backing field for you	<code>public int Id { get; }</code>
<code>private / public</code>	Access modifiers — who can see a member	private field, public property
<code>this</code>	A reference to the current object	<code>this.Id = id;</code>
<code>ToString()</code> override	Lets an object describe itself when printed	<code>public override string ToString()</code>
encapsulation	Keeping data private and exposing it only through guarded members	private fields + validating properties

Self-study links:

- C# classes and objects: https://www.w3schools.com/cs/cs_classes.php
- C# constructors: https://www.w3schools.com/cs/cs_constructors.php
- C# properties: https://www.w3schools.com/cs/cs_properties.php
- C# access modifiers: https://www.w3schools.com/cs/cs_access_modifiers.php
- Short video — C# OOP in 100 seconds: <https://www.youtube.com/watch?v=pTB0EiLXUC8>

Self-Investigation (10:30 – 11:30)

Answer in your own words — no code needed yet.

1. Last week your track was four parallel lists. What exact bug does putting id, speed, and heading into one object prevent? Describe a sequence of actions that breaks the parallel lists but cannot break the object.
2. A constructor runs "when the object is created." How could you *prove* to yourself, with one line of code, that it actually ran?

3. The moment you write your own constructor, `new Track()` (with no arguments) stops compiling. Why do you think the language takes the free one away?
 4. What problem does `this(...)` chaining solve? Imagine two constructors that both need to validate a heading — what goes wrong if you copy that validation into both?
 5. A field is `private` and its property checks the value before storing it. What exactly is being protected, and from whom? How does this relate to last week's "anyone could overwrite the heading variable"?
-

Code Examples (10:30 – 11:30)

Example 1 — A class, an object, and the constructor firing

```
class Track
{
    public int Id;           // a field (we will make this private soon)
    public double Speed;

    public Track(int id, double speed) // constructor: runs on `new`
    {
        Id = id;
        Speed = speed;
        Console.WriteLine("constructor ran"); // proof it fired
    }
}

// create two objects from the one blueprint:
Track a = new Track(17, 412.5);
Track b = new Track(8, 95.0);
Console.WriteLine($"{a.Id} at {a.Speed} km");
```

Line by line:

- `class Track { ... }` is the blueprint. It is not a track yet.
- `new Track(17, 412.5)` builds one object and runs the constructor — you will see `constructor ran` print.
- `a` and `b` are two separate objects from the same class, each with its own data.

How to run: `dotnet new console -n Day1`, put the class and the calls in `Program.cs`, `dotnet run`. Watch how many times `constructor ran` prints.

Example 2 — The default constructor disappears

```
class Track
{
    public int Id;
}
```

```
// With NO constructor written, this works for free:  
Track t = new Track();    // the "default constructor"
```

Now add one parameterized constructor:

```
class Track  
{  
    public int Id;  
    public Track(int id) { Id = id; }  
}  
  
// Track t = new Track();    // <- now this FAILS to compile  
Track t = new Track(17);    // you must use the one you wrote
```

Line by line:

- An empty class gives you a free parameterless constructor.
- The instant you write any constructor, that free one is gone.
- If you still want `new Track()`, you have to add a parameterless constructor back yourself.

How to run: try `new Track()` after adding the parameterized constructor and read the compile error. That error is the lesson.

Example 3 — Overloading and chaining with `this(...)`

```
class Track  
{  
    public int Id;  
    public double Speed;  
    public double Heading;  
  
    // full constructor – the one real setup lives here  
    public Track(int id, double speed, double heading)  
    {  
        Id = id;  
        Speed = speed;  
        Heading = heading;  
    }  
  
    // overloaded + chained: forwards to the full one, no duplicated setup  
    public Track(int id) : this(id, 0.0, 0.0) { }  
}  
  
Track full = new Track(17, 412.5, 270);  
Track quick = new Track(8);    // speed and heading default to 0
```

Line by line:

- Two constructors share the name `Track` but take different parameters — that is **constructor overloading**, the same idea as last week's method overloading, in a new place.
- `: this(id, 0.0, 0.0)` makes the short constructor call the full one. The setup is written **once**.
- `new Track(8)` ends up running the full constructor with defaults filled in.

How to run: create both objects, print their fields, confirm `quick` has speed and heading `0`.

Example 4 — Encapsulation: private fields + validating properties + ToString

```
class Track
{
    private double _heading; // private field – guarded

    public int Id { get; } // read-only after
construction
    public double Speed { get; set; }
    public double Heading // property with validation
    {
        get => _heading;
        set
        {
            if (value < 0 || value > 359)
                _heading = 0; // correct an invalid value
            else
                _heading = value;
        }
    }

    public Track(int id, double speed, double heading)
    {
        Id = id;
        Speed = speed;
        Heading = heading; // goes through the
validating setter
    }

    public override string ToString() // the object prints itself
    => $"Track {Id}: {Speed} kn, heading {Heading}";
}

Track t = new Track(17, 412.5, 270);
Console.WriteLine(t); // ToString() is called automatically
t.Heading = 999; // invalid
Console.WriteLine(t.Heading); // 0 – the setter corrected it
```

Line by line:

- `_heading` is **private** — no outside code can corrupt it directly.
- The `Heading` property checks the value in its `set` before storing it.

- `public int Id { get; }` has no setter, so `Id` cannot change after the object is built.
- `Console.WriteLine(t)` calls `ToString()` for you — the object describes itself.
- Setting `Heading = 999` is rejected and corrected to `0`. **That guard is encapsulation.**

How to run: print the track, set an invalid heading, print again, and confirm it was corrected.

Comprehension Questions (10:30 – 11:30)

1. (multiple choice) What is the difference between a class and an object? a) nothing, they are synonyms
b) a class is the blueprint, an object is built from it c) an object is the blueprint d) a class can only make one object
 2. (multiple choice) When does a constructor run? a) once when the program starts b) every time you call any method c) when you create the object with `new` d) never automatically
 3. (multiple choice) You write `public Track(int id) { ... }`. What happens to `new Track()` (no arguments)? a) still works b) fails to compile unless you add a parameterless constructor c) returns null d) runs the int constructor with 0
 4. (open) Explain constructor chaining with `this(...)` in your own words. What does it save you from?
 5. (open) A field is private and its property validates input. Describe, with last week's parallel-list code in mind, exactly what encapsulation is protecting.
-

Coding Exercise (11:30 – 12:45 planning, 14:45 – 17:45 build)

Before writing any code: write a plan on paper. Decide what one thing your class represents, list its fields, mark which are private, decide which properties validate and what they reject, and sketch your constructors (which one does the real work, which ones chain to it). Only open the editor once the plan is written.

Title: One Well-Encapsulated Class

Build a console program whose central artifact is **one class** that bundles together data which last week would have lived in parallel lists, and hold many of these objects in a `List<>`.

Functional requirements:

- Define one class with at least three related fields that belong together (replacing parallel lists).
- All data fields are **private**; expose them through properties.
- At least one property must **validate** its input in the `set` (reject or correct an out-of-range value).
- Provide at least one property that is read-only after construction (no public setter).
- Write **at least two constructors**: one full parameterized constructor and one shorter constructor that **chains** to it with `this(...)`.
- Override `ToString()` so an object prints a clean one-line readout of itself.
- Hold many of these objects in a `List<>`, add a few, and print them all in a loop — show that the parallel-list pattern from last week is gone.

You may pick any domain you like — what matters is the content above, not the subject.

This exercise must stand on its own — do not copy the examples above. Use only their ideas.

Common Errors

Error	Cause	Fix
Track does not contain a constructor that takes 0 arguments	Wrote a parameterized constructor, then tried <code>new Track()</code>	Add a parameterless constructor, or pass the arguments
Setting an invalid value still "works"	Used a public field instead of a validating property	Make the field private; validate in the property's <code>set</code>
Every object shows the same data	Fields declared <code>static</code> by accident, or set outside the object	Per-object data is a normal (non-static) field set in the constructor
<code>ToString()</code> does not change the output	Forgot <code>override</code> , or wrong signature	It must be exactly <code>public override string ToString()</code>
Stack overflow / constructor loops forever	A constructor chained to itself	<code>this(...)</code> must forward to a <i>different</i> constructor
Can change a value that should be fixed	Gave a property a public <code>set</code> you did not want	Use <code>{ get; }</code> or <code>{ get; private set; }</code>

Self-Assessment

Skill	Status
I can define a class and create objects from it with <code>new</code>	
I can explain the difference between a class and an object	
I can write a constructor and prove when it runs	
I can use constructor overloading and <code>this(...)</code> chaining	
I can make fields private and guard them with validating properties	
I can override <code>ToString()</code> so an object prints itself	
I can explain encapsulation and point to where I built it	

Summary and Tomorrow's Task (17:15 – 17:45)

Today the parallel lists became one object. You wrote a class, gave it a constructor that builds it correctly in one step, guarded its data with private fields and validating properties, and let it describe itself with `ToString()`. The name for what you built is **encapsulation** — data and the rules that protect it, living together.

Task before tomorrow: You now have one kind of object. But imagine some of your objects need an extra piece of data the others don't, and each kind should describe itself differently. Write two sentences: would you

write three completely separate classes and repeat the shared fields in each, or is there a better way to share what's common? Tomorrow we answer that.